

1. Programs and the log

A signature of operations, a program as a tree over it, and the events of the log.

```
-- A signature: the operations a program may perform, each with the type of its answer.
Hancock and Setzer (2000) call it a world, of commands and responses; McBride (2015) has the
same pair as `S` and `T`. -/
structure Signature where
  Op : Type
  Answer : Op → Type

-- A frame is the path of calls from the root, each call being its ordinal among the calls
its parent made. Nothing in a program states an ordinal: the interpreter assigns it. `#[ ]` is
the frame of a run, and `#[0]` in it that of the agent. -/
abbrev Frame := Array Nat

-- Something that happened without a program asking for it, or the reply of a person to a
question a program asked: that is asked for, but it comes in its own time, from outside, so it
is logged when it arrives like the rest. A reply names the call that asked. -/
inductive Notice where
| said (message : String) -- a person spoke
| changed (workspace : Snapshot) (summary : String) -- the workspace was changed from outside
| replied (to : Frame) (reply : Reply) -- a person answered a question

def Notice.isReply : Notice → Bool
| .replied .. => true
| _ => false

-- A program: a tree of operations, each continued with its answer, or with the error when the
world could not give one. It is the free monad on its signature (`Program` in Apfelmus 2010;
Kiselyov and Ishii 2015), with three constructors more, and a way to fail. The inbox gives the
notices not yet read, replies apart. A read that waits is for some notices only, which it says
given the frame it is made in: it is made once one of them has arrived, takes those that stand
at the end of the log, and leaves the rest unread. A call names a tool and what it is called with, and
holds no body: it is a request, as a recursive call is in McBride (2015), and so it is data.
The interpreter answers it by running the tool the run has under that name, in a child frame.
A program cannot run a tool otherwise, and the opening the log holds is the call itself. A
program that fails gives up what it was doing, up to the call it is in: a call ends with the
value of its tool or with its error (the `Catch` of Wu, Schrijvers and Hinze 2014). A loop
goes round `step` from a state until a round gives a result: `iter` of Xia et al. (2020),
`while` of Hancock and Setzer (2000). The state of a loop is data, where a continuation is
not. -/
inductive Program (σ : Signature) : Type → Type 1 where
| pure : α → Program σ α
| fail : (error : String) → Program σ α -- give up
| perform : (op : σ.Op) → (Except String (σ.Answer op) → Program σ α) → Program σ α -- ask
| inbox : (wait : Option (Frame → Notice → Bool)) → (List Notice → Program σ α) → Program σ α
| call : ToolCall → (Except String Json → Program σ α) → Program σ α -- a tool, by its name
| iter : {S : Type} → (S → Program σ (S ⊗ β)) → S → (β → Program σ α) → Program σ α

-- Substitution. `perform` commutes with it, which is what makes an operation algebraic
(Plotkin and Power 2003), and so do a read of the inbox, a call, and `fail`, which has nothing
to continue. A loop does not: only its continuation is extended, never its rounds. -/
def Program.bind : Program σ α → (α → Program σ β) → Program σ β
| .pure a, f => f a
| .fail error, _ => .fail error
| .perform op k, f => .perform op fun answer => (k answer).bind f
| .inbox wait k, f => .inbox wait fun notices => (k notices).bind f
| .call tool k, f => .call tool fun result => (k result).bind f
| .iter step s k, f => .iter step s fun b => (k b).bind f

instance : Monad (Program σ) := { pure := .pure, bind := .bind }

-- A program made to give its value or its failure: what `try` and `catch` are. Raising is
algebraic, and handling is "of a different computational character" (Plotkin and Power 2003):
here it is a function of the whole program. It does not reach into a tool that is called,
whose failure the call has caught already. Nothing is logged for it. -/
def Program.attempt : Program σ α → Program σ (Except String α)
| .pure a => .pure (.ok a)
| .fail error => .pure (.error error)
| .perform op k => .perform op fun answer => (k answer).attempt
| .inbox wait k => .inbox wait fun notices => (k notices).attempt
| .call tool k => .call tool fun result => (k result).attempt
| .iter step s k =>
  .iter (fun s => (step s).attempt.bind fun
    | .ok (.inl s) => .pure (.inl s)
    | .ok (.inr b) => .pure (.inr (Except.ok b))
    | .error error => .pure (.inr (Except.error error))) s fun
  | .ok b => (k b).attempt
  | .error error => .pure (.error error)

instance : MonadExcept String (Program σ) where
  throw := .fail
  tryCatch body handler := body.attempt.bind fun
  | .ok a => .pure a
  | .error error => handler error

def inbox : Program σ (List Notice) := .inbox none .pure
def await (accepts : Frame → Notice → Bool) : Program σ (List Notice) := .inbox (some accepts) .pure
-- Tries a program again while it fails, up to `attempts` times more. Every try is in the log,
with how it ended. -/
def retry (attempts : Nat) (program : Program σ α) : Program σ α :=
  match attempts with
  | 0 => program
  | attempts + 1 => try program catch _ => retry attempts program

-- Calls a tool. Its failure is its caller's too, unless the caller catches it. -/
def call (name : String) (arguments : Json) : Program σ Json :=
  .call {name, arguments} fun | .ok value => .pure value | .error error => .fail error

-- The tools of a run, by name. -/
abbrev Tools (σ : Signature) := String → Option (Json → Program σ Json)
def iter (step : S → Program σ (S ⊗ α)) (s : S) : Program σ α := .iter step s .pure

-- What the driver is asked to do: an operation, and the frame that asked. -/
structure Call (σ : Signature) where
  frame : Frame
  op : σ.Op

-- The log is a list of events. A call with its answer is what the program asked for; the
answer is an error when the world could not give one, and the call then fails. A notice is
not asked for: the driver logs it when it arrives, before anything acts on it, as Goldstein et al.
(2020) do with input that cannot be replayed. The other three are marks of what the program
did, logged so that the log can be read without the program. A read of the inbox holds the
positions in the log of the notices it took.
A call is bracketed by its opening, which is the call, and its end: a return, with what the
tool gave, or a failure, with the error. They are the begin and end markers of Wu, Schrijvers and
Hinze (2014, §7). The frame `#[ ]` of the run
has a return alone, and a log is complete when it ends with it. A stop, like a notice, comes
from outside; it is not read: it ends every frame of the agent. -/
inductive Event (σ : Signature) where
| arrived (notice : Notice)
| heard (frame : Frame) (notices : List Nat)
| answered (call : Call σ) (answer : Except String (σ.Answer call.op))
| opened (frame : Frame) (tool : ToolCall)
| returned (frame : Frame) (value : Json)
| failed (frame : Frame) (error : String)
| stopped (reason : String)
```

2. Operations, tools, sub-agents and the agent

The agent's operations, and the tools of a run, which programs call by name: commands, questions, sub-agents and the agent itself.

```
-- What an agent may ask for. -/
inductive Op where
| sample (request : Request) -- answered by the model, which may fail to
| exec (command : String) -- answered by the executor; changes the workspace
| time -- answered by the clock
-- An external program, opaque to the run: `command` in a fresh container of `image`, with
no network, on a checkout of the workspace, and with `input`, files that are not in the
workspace, mounted to read. The checkout it leaves is in its answer; the workspace of the
run stays where it is. -/
| external (command image : String) (input : Option Snapshot) (timeout : Nat)
deriving DecidableEq

def Op.Answer : Op → Type
| .sample _ => Response
| .exec _ => Output
| .time => Nat × Option Nat
| .external .. => ExternalOutput

abbrev Agent : Signature := (Op, Op.Answer)

-- The generic effect of an operation (Plotkin and Power 2003): perform it and return. An
operation the world could not answer fails, where it was performed. -/
def perform (op : Op) : Program Agent op.Answer :=
  .perform op fun | .ok answer => .pure answer | .error error => .fail error

-- A tool is a program from a call's arguments to its result, under a name. Programs call a
```

```
tool by its name alone, so it always runs in a frame of its own. -/
structure Tool where
  name : String
  run : Json → Program Agent Json

/-- Runs a command. A command that exits with an error is a failure of the tool. -/
def bash : Tool where
  name := "bash"
  run command := do
    let output ← perform (.exec command)
    if output.exit != 0 then throw s!"exit {output.exit}: {output.text}"
    return output.text

def timeBudget : Tool where
  name := "time_budget"
  run _ := do
    let (spent, budget?) ← perform .time
    return match budget? with
      | some budget => s!"{(budget - spent) / 1000} s left"
      | none => "no limit"

/-- What a notice says. -/
def Notice.text : Notice → String
| .said message => message
| .changed _ summary => s!"The workspace was changed: {summary}"
| .replied _ reply => reply.render

/-- Asks a person. The question is the argument, so the opening of the call puts it in the
log, with the form its answer is to have. The tool then waits for a reply to this call, of
that form: it is logged when the person gives it, and no other notice ends the wait or is
taken for the answer. So the run stops at a question as a script of Thiemann (2002) stops at a
form until the user responds. -/
def askUser : Tool where
  name := "ask_user"
  run arguments := do
    let question ← match Question.parse arguments with
      | .ok question => pure question
      | .error problem => throw problem
    let replies ← await fun frame notice =>
      match notice with
      | .replied to reply => to == frame && question.accepts reply
      | _ => false
    return "\n".intercalate (replies.map (·.text))

/-- One round of a conversation with the model: sample, and call each tool the response asks
for, by its name. The conversation offers its model some of the tools of the run, and calls no
other. A model that fails to answer is asked again, as often as the agent's settings say. A
tool that fails does not fail the conversation: the model is told of the error, as the result
of its call. The round gives the longer dialogue, or the answer when a response makes no
calls. A conversation that listens reads the inbox at the end of each round, for the model to
hear of it in the next. -/
def round (agent : AgentConfig) (model : ModelConfig) (listen : Bool) (dialogue : Dialogue) :
  Program Agent (Dialogue × String) := do
  let request := { model, messages := dialogue, tools := agent.tools }
  let response ← retry agent.retries (perform (.sample request))
  if response.toolCalls.isEmpty then return .inr response.text
  let mut results := []
  for asked in response.toolCalls do
    let result ←
      if agent.tools.contains asked.name then
        try call asked.name asked.arguments catch error => pure s!"error: {error}"
      else pure s!"error: {asked.name} is not a tool of this conversation"
    results := results ++ [.tool result]
  let heard ← if listen then inbox else pure []
  return .inl (dialogue ++ [.assistant response.text response.toolCalls] ++ results
    ++ heard.map (.user ·.text))

/-- A conversation is a loop of rounds whose state is the dialogue. Every round samples, so
the loop is guarded. A model's answers are the random choices of this program, and a log is
one trace of it (Dohan et al. 2022). -/
def converse (agent : AgentConfig) (model : ModelConfig) (dialogue : Dialogue)
  (listen := false) : Program Agent String :=
  iter (round agent model listen) dialogue

/-- A sub-agent is a tool whose program is a conversation of its own, with the model of the
agent that started the run. Its samples and its tools' frames nest inside the call's frame;
the outer model sees only what it returns. -/
def delegate (model : ModelConfig) : Tool where
  name := "delegate"
  run task := converse { tools := ["bash", "time_budget"], retries := 2 } model
    [.system "You are a sub-agent.", .user task]

/-- A tool that calls tools: a sub-agent runs the tests, and then two commands commit the work.
Each of the three calls opens a frame in this tool's frame, so frames nest as the calls do. -/
def commit : Tool where
  name := "commit"
  run message := do
    let report ← call "delegate" "run the tests"
    let _ ← call "bash" "git add"
    let _ ← call "bash" s!"git commit -m {message}"
    return report

/-- The agent is the outermost tool. It is called with its configuration: its system message,
its settings, and those of its model. It cannot start without a task, so it first waits for
one: a notice, like all that a person says. It is the only conversation that listens. What it
returns is its submission. The stub keeps the configuration it is built from, where a real
agent would read it from its call. -/
def agent (config : Config) : Tool where
  name := "agent"
  run _ := do
    let task ← await fun _ notice => notice.matches .said _
    converse config.agent config.model ([.system config.system] ++ task.map (.user ·.text))
    (listen := true)

/-- The tools of a run, by name, from a list of them. -/
def table (tools : List Tool) : Tools Agent :=
  fun name => (tools.find? (·.name == name)).map (·.run)
```

3. Replay

The interpreter: it turns a run, the agent and what follows it, into the pure, total function from the log that the driver sees.

```
/-- Where replay has got to: how many events it has passed, the notices among them that the
program has not read, each with its position in the log, and how many calls the current
frame has opened (the next one's ordinal). -/
structure Cursor where
  position : Nat := 0
  unread : List (Nat × Notice) := []
  opened : Nat := 0

/-- What to do next, as far as the log tells. A log that is no trace of the program is a
result of its own, not an outcome of the agent. -/
inductive Next (σ : Signature) (α : Type) where
| done (a : α)
| ask (call : Call σ) -- the first operation the log has no answer to
| hears (frame : Frame) (notices : List Nat) -- the first read of the inbox not yet marked
| waits (frame : Frame) -- a read that waits, and nothing has arrived for it
| opens (frame : Frame) (tool : ToolCall) -- the first call whose opening is not logged
| returns (frame : Frame) (value : Json) -- the first call to end with its return not logged
| fails (frame : Frame) (error : String) -- the first call to fail with that not logged
| raised (error : String) (cursor : Cursor) -- a failure, on its way to the call that catches
| stopped (cursor : Cursor) -- the run was stopped from outside: every frame ends
| mismatch (position : Nat) -- the event here is not what the program does now
| unguarded (frame : Frame) -- a loop went round without reading an event

def Next.bind : Next σ α → (α → Next σ β) → Next σ β
| .done a, f => f a
| .ask call, _ => .ask call
| .hears frame notices, _ => .hears frame notices
| .waits frame, _ => .waits frame
| .opens frame tool, _ => .opens frame tool
| .returns frame value, _ => .returns frame value
| .fails frame error, _ => .fails frame error
| .raised error cursor, _ => .raised error cursor
| .stopped cursor, _ => .stopped cursor
| .mismatch position, _ => .mismatch position
| .unguarded frame, _ => .unguarded frame

instance : Inhabited (Next σ α) := ⟨.mismatch 0⟩

/-- The notices at the front of the events, the first of them at `position` in the log. -/
def notices : Nat → List (Event σ) → List (Nat × Notice)
| position, .arrived notice :: rest => (position, notice) :: notices (position + 1) rest
| _, _ => []

/-- Passes the notices at the cursor and keeps them for the next read of the inbox. They are
events of the environment, taken in until it is the program's turn (Gu et al. 2018). So a
notice is delivered by where it stands in the log: to the first read after it that takes it. -/
def Cursor.pass (cursor : Cursor) (log : List (Event σ)) : Cursor :=
  let passed := notices cursor.position (log.drop cursor.position)
```

```

{ cursor with position := cursor.position + passed.length, unread := cursor.unread ++ passed }

/-- The answer the event gives to `call`, which may be an error. The check that it answers
exactly this call is not needed to replay a log (Thiemann 2002 logs answers alone); it is what
detects a program that is not the one that wrote the log. -/
def Event.answer? [DecidableEq  $\sigma$ .Op] (call : Call  $\sigma$ ) :
  Event  $\sigma$  → Option (Except String  $\sigma$ .Answer call.op))
| .answered logged answer =>
  if h : logged.frame = call.frame ∧ logged.op = call.op then some (h.2 ▶ answer) else none
| _ => none

/-- Reads a mark off the log: an event that says what the program did, with nothing in it for
the program to learn. `missing` is what to do when the log ends before it. Replay knows what
the mark must be and checks the logged one against it, so the brackets of a call cannot be
out of place, where markers alone "run the risk of being unbalanced" (Wu, Schrijvers and Hinze
2014). -/
def Cursor.mark (cursor : Cursor) (log : List (Event  $\sigma$ )) (expected : Event  $\sigma$  → Bool)
  (missing : Next  $\sigma$  Cursor) : Next  $\sigma$  Cursor :=
  let cursor := cursor.pass log
  let after := { cursor with position := cursor.position + 1 }
  match log[cursor.position]? with
  | none => missing
  | some (.stopped _) => .stopped after
  | some event => if expected event then .done after else .mismatch cursor.position

/-- Ends a frame: reads off the log the mark of how its body ended, a return with its value
or a failure with its error. This is where a failure is caught. -/
def close (log : List (Event  $\sigma$ )) (frame : Frame) :
  Next  $\sigma$  (Json × Cursor) → Next  $\sigma$  (Except String Json × Cursor)
| .raised error cursor =>
  let failing | .failed left given => left == frame && given == error | _ => false
  (cursor.mark log failing (.fails frame error)).bind fun cursor => .done (.error error, cursor)
| body => body.bind fun (value, cursor) =>
  let returning | .returned left given => left == frame && given == value | _ => false
  (cursor.mark log returning (.returns frame value)).bind fun cursor => .done (.ok value, cursor)

/-- Goes round a loop until a round gives a result. A round must read an event, so the length
of the log bounds the rounds: it is the fuel (the petrol of McBride 2015), and the guard is the
one that makes a loop well defined (Hancock and Setzer 2000; Piróg and Gibbons 2014). -/
def rounds (round : S → Cursor → Next  $\sigma$  ((S ⊗  $\beta$ ) × Cursor)) (frame : Frame) :
  Nat → S → Cursor → Next  $\sigma$  ( $\beta$  × Cursor)
| 0, _, _ => .unguarded frame
| fuel + 1, s, cursor =>
  (round s cursor).bind fun
  | (.inr b, after) => .done (b, after)
  | (.inl s, after) =>
    if after.position > cursor.position then rounds round frame fuel s after
    else .unguarded frame

/-- Calls a tool in the frame `child`: reads its opening off the log, lets `enter` run its body,
and reads how it ended. -/
def invoke (enter : ToolCall → Frame → Cursor → Next  $\sigma$  (Json × Cursor)) (log : List (Event  $\sigma$ ))
  (tool : ToolCall) (child : Frame) (cursor : Cursor) : Next  $\sigma$  (Except String Json × Cursor) :=
  let opening | .opened entered called => entered == child && called == tool | _ => false
  (cursor.mark log opening (.opens child tool)).bind fun inside =>
  close log child (enter tool child { inside with opened := 0 })

/-- Runs `program` against the log, from the start: every answer is read back in order, to
continue the program where it stopped (Thiemann 2002; Koppel, Scherer and Solar-Lezama 2018;
Burckhardt et al. 2021). It is a handler that passes its place in the log along (Plotkin and
Pretnar 2009, §6.7). A call is handled by entering the tool it names in a child frame, with a
counter of its own, and going on in the parent's (the promotion and demotion of Piróg et al.
2018); its opening and its end are marks in the log. -/
def replay [DecidableEq  $\sigma$ .Op] (enter : ToolCall → Frame → Cursor → Next  $\sigma$  (Json × Cursor))
  (log : List (Event  $\sigma$ )) : { $\alpha$  : Type} → Program  $\sigma$   $\alpha$  → Frame → Cursor → Next  $\sigma$  ( $\alpha$  × Cursor)
| _, .pure a, _, cursor => .done (a, cursor)
| _, .fail error, _, cursor => .raised error cursor
| _, .perform op k, frame, cursor =>
  let cursor := cursor.pass log
  match log[cursor.position]? with
  | none => .ask { frame, op }
  | some (.stopped _) => .stopped { cursor with position := cursor.position + 1 }
  | some event =>
    let cursor := { cursor with position := cursor.position + 1 }
    match event.answer? { frame, op } with
    | some answer => replay enter log (k answer) frame cursor
    | none => .mismatch (cursor.position - 1)
| _, .inbox wait k, frame, cursor =>
  let arrived := notices cursor.position (log.drop cursor.position)
  -- what the read takes, and what it leaves unread: a read that waits takes those of the
  -- notices just arrived that it is for; any other takes all that is unread, replies apart
  let (taken, left) : List (Nat × Notice) × List (Nat × Notice) := match wait with
  | some accepts =>
    let (mine, others) := arrived.partition fun (_, notice) => accepts frame notice
    (mine, cursor.unread ++ others)
  | none => (cursor.unread ++ arrived).partition fun (_, notice) => !notice.isReply
  let passed := cursor.pass log
  if wait.isSome ∧ taken.isEmpty then
    match log[passed.position]? with
    | none => .waits frame
    | some (.stopped _) => .stopped { passed with position := passed.position + 1 }
    | some _ => .mismatch passed.position
  else
    let positions := taken.map (·.1)
    let reading | .heard reader read => reader == frame && read == positions | _ => false
    (cursor.mark log reading (.hears frame positions)).bind
    fun cursor => replay enter log (k (taken.map (·.2))) frame { cursor with unread := left }
| _, .call tool k, frame, cursor =>
  (invoke enter log tool (frame.push cursor.opened) cursor).bind fun (result, after) =>
  replay enter log (k result) frame { after with opened := cursor.opened + 1 }
| _, .iter step s k, frame, cursor =>
  (rounds (fun s => replay enter log (step s) frame) frame (log.length + 1) s cursor).bind
  fun (b, cursor) => replay enter log (k b) frame cursor

/-- Enters a tool: looks its name up among the tools of the run and replays its body. A name
the run does not have is a failure of the call. The body is not part of the program that
called it, so this is a recursion of its own, on fuel that bounds how deep calls nest. A body
is entered only once the log holds the opening of its call, so the length of the log is fuel
enough, as it is for a loop (McBride 2015). -/
def enter [DecidableEq  $\sigma$ .Op] (tools : Tools  $\sigma$ ) (log : List (Event  $\sigma$ )) :
  Nat → ToolCall → Frame → Cursor → Next  $\sigma$  (Json × Cursor)
| 0, _, frame, _ => .unguarded frame
| fuel + 1, tool, frame, cursor =>
  match tools tool.name with
  | some body => replay (enter tools log fuel) log (body tool.arguments) frame cursor
  | none => .raised s!"no tool named {tool.name}" cursor

/-- A run: the tools it has, the call of the agent among them, and what follows the agent.
What follows is given what the agent returned, or the error when it failed or was stopped from
outside. -/
structure Run ( $\sigma$  : Signature) where
  tools : Tools  $\sigma$ 
  call : ToolCall
  after : Except String Json → Program  $\sigma$  Json

/-- A run, as the driver sees it: a pure function from the log to what to do next. A
participant as "a deterministic partial function from the current log to its next move" is the
strategy of Gu et al. (2018).

```

A run starts from a root: the first event of its log is a change from outside, the workspace that someone provides. Without it the run waits. The root is the ground the run stands on and not a notice for the agent: no read of the inbox takes it.

The agent is called in the frame `#[0]`, and what follows it runs in `#[1]`. A stop ends every frame of the agent, whatever the nesting, and nothing in the agent can catch it, where a failure is caught by the call it happens in. What the agent did is kept in either case, since the log only grows. So what follows the agent runs when the agent is over, and nothing it does can reach the agent. The run is done once the log holds how it ended; an event left over after that is a mismatch too. -/

```

def next [DecidableEq  $\sigma$ .Op] (run : Run  $\sigma$ ) (log : List (Event  $\sigma$ )) : Next  $\sigma$  Json :=
  match log with
  | [] => .waits #[]
  | .arrived (.changed _ _) :: _ =>
    let enter := enter run.tools log (log.length + 1)
    let ended : Next  $\sigma$  (Except String Json × Cursor) :=
      match invoke enter log run.call #[0] { position := 1 } with
      | .stopped cursor => .done (.error "stopped", cursor)
      | agent => agent
    let whole := ended.bind fun (result, cursor) =>
      close log #[1] (replay enter log (run.after result) #[1] { cursor with opened := 1 })
    match whole with
    | .stopped cursor => .mismatch (cursor.position - 1) -- a stop when the agent was over
    | whole => whole.bind fun (result, cursor) =>
      let cursor := cursor.pass log
      if cursor.position < log.length then .mismatch cursor.position
      else match result with
      | .ok value => .done value
      | .error error => .raised error cursor
  | _ => .mismatch 0

```

4. The driver

Its loop: log what has arrived, then carry out a call and log its answer, or log a mark: a read of the inbox, a call that opens, a call that returns or fails.

```
-- The version of the workspace the log has reached. A command leaves a version, and so does
a change from outside (the time-stamped handles of Thiemann 2002); the first is that of the
root. An external program works on a checkout of its own, which is not the run's workspace. Without the versions a command would depend on
Mitchell and Peyton Jones 2018). -/
def workspace (log : List (Event Agent)) : Snapshot :=
  log.foldl (init := "") fun version event =>
    match event with
    | .answered (_, .exec _) (.ok output) => output.workspace
    | .arrived (.changed workspace _) => workspace
    | _ => version

-- A person's reply to the question a log waits on, as the event to append. It is refused when
no question waits, or when the reply is not of the form the question asks for: the question is
read off the opening of the call that asked. -/
def reply (log : List (Event Agent)) (answer : Reply) : Except String (Event Agent) := do
  let bracket := log.reverse.find? fun
    | .opened .. | .returned .. | .failed .. | .stopped _ => true
    | _ => false
  match bracket with
  | some (.opened frame ("ask_user", arguments)) =>
    if (← Question.parse arguments).accepts answer then pure (.arrived (.replied frame answer))
    else throw "the reply is not of the form the question asks for"
  | _ => throw "no question waits for a reply"

-- The world outside the program. A real one is a model, an executor, a clock and people; the
test scripts one. A model may fail to answer, and what it does can change with time, for which
the log stands. -/
structure World where
  model : List (Event Agent) → Request → Except String Response
  execute : Snapshot → String → Output -- runs a command on a version of the workspace
  -- Runs an external program, in its image, on a checkout of a version of the workspace. -/
  external : Snapshot → String → String → ExternalOutput
  clock : Nat × Option Nat
  arrivals : List (Event Agent) → List Notice -- what has happened, unasked, since the log

-- Carries out an operation: the handler (Plotkin and Pretnar 2009). A command runs on the
version the log has reached, which the executor restores if the workspace is at another; so a
command that is run again after a crash starts from the same workspace. No more than that is
promised: a command happens at least once, and an effect of it beyond the workspace may happen
twice (Zhang et al. 2020; Burckhardt et al. 2021). The answer is an error when the world could
not give one. A driver may well try again by itself first, which leaves nothing in the log; an
error it gives up on is logged as the answer, and it is then for the program to deal with. -/
def World.answer (world : World) (log : List (Event Agent)) :
  (op : Op) → Except String op.Answer
  | .sample request => world.model log request
  | .exec command => .ok (world.execute (workspace log) command)
  | .time => .ok world.clock
  | .external command image _ _ => .ok (world.external (workspace log) image command)

-- The driver. "Execution is an external operation rather than a constant within type theory"
(Hancock and Setzer 2000), so this is the one definition that need not terminate. Whatever has
happened unasked is logged first, the root before all. Then the call the log has no answer to
is carried out and its answer logged; or a mark is logged, of a read of the inbox or of a call
that opens or ends. When the program waits for a notice and none has arrived, the driver stops:
there is nothing to do, and this same loop resumes the run once the log holds one. Resuming
after a crash is no different: a call that was being carried out is asked for again. -/
partial def drive (world : World) (run : Run Agent) (log : List (Event Agent)) :
  List (Event Agent) × Next Agent Json :=
  let log := log ++ (world.arrivals log).map .arrived
  match next run log with
  | .ask call => drive world run (log ++ [.answered call (world.answer log call.op)])
  | .hears frame notices => drive world run (log ++ [.heard frame notices])
  | .opens frame tool => drive world run (log ++ [.opened frame tool])
  | .returns frame value => drive world run (log ++ [.returned frame value])
  | .fails frame error => drive world run (log ++ [.failed frame error])
  | result => (log, result)
```

5. Grading

What follows the agent in a run: the graders. A grader runs an external program, in a container of its own, and reads its verdict. A state of the run is graded on a fork of the log that is stopped there.

```
-- A verdict, read off what a grader prints: how many of its checks passed, of how many, in the
Test Anything Protocol's lines `ok` and `not ok`. -/
def verdict (stdout : String) : String :=
  let lines := stdout.splitOn "\n"
  let passed := (lines.filter (·.startsWith "ok")).length
  let failed := (lines.filter (·.startsWith "not ok")).length
  s!"{if failed == 0 then "pass" else "fail"} {passed}/{passed + failed}"

-- A grader is a tool that no conversation offers its model, and needs nothing of its own: it
is an external program and a reading of what it prints. This one runs tests the agent never
sees: in an image of its own, on a checkout of the workspace as the agent left it, with the
tests mounted to read. The operation holds all of that, so the log does. -/
def hiddenTests : Tool where
  name := "hidden_tests"
  run _ := do
    let ran ← perform (.external "pytest /grader" "grader@sha256:9f2c" (some "hidden") 900)
    return verdict ran.stdout

-- What follows the agent in a run: the graders. They are given what the agent returned, or
the error when it failed or was stopped, and what they return is the result of the run. The
agent is over when they start, so whatever they do, the agent cannot see it. To grade a state
a run went through is no other thing: the log is forked there and stopped, and the graders
follow. -/
def grading (_ : Except String Json) : Program Agent Json := call "hidden_tests" ""

-- A run of the agent with its graders. The agent is called with its configuration, which
the log so holds from the start. The graders are among the tools of the run, and no
conversation offers them to a model. -/
def graded (config : Config) : Run Agent :=
  { tools := table
    [agent config, bash, timeBudget, askUser, delegate config.model, commit, hiddenTests]
    call := ("agent", config.json)
    after := grading }
```

```
-- The positions at which the workspace is at a version for the first time. Graders that look
at the workspace alone give one grade for a version, so these are all the states there is to
grade: a result kept by what it was computed from, as Mokhov, Mitchell and Peyton Jones (2018)
keep the results of a build. -/
def versions (log : List (Event Agent)) : List Nat :=
  let reached := fun position => workspace (log.take position)
  (List.range (log.length + 1)).filter fun position =>
    position > 0 && reached position != reached (position - 1)
```

6. Related work

Twenty-five papers read against the sketch: what corresponds, what differs, and what to take. Section numbers are the papers’ own; the PDFs are in [literature/](#).

Programs as trees of operations

Hancock and Setzer (2000) is the closest ancestor. A *world* is a set C of commands with a family $R : C \rightarrow \text{Set}$ of responses, which is the sketch’s Signature. An I/O-tree is built from leaf and `do c p`, which are pure and perform; their bind is the sketch’s, clause for clause; and “execution is an external operation rather than a constant within type theory”, which is the driver. They call such a tree a strategy for one party in a command/response interaction, the reading that next makes literal. Their §3 writes loops by general recursion over well-founded trees, and names the price: definitional equality and type checking become undecidable. In Lean the same route needs `partial`, which makes a definition opaque to proof. Their §4 regains normalisation by making the loop a constructor (`while`) whose body must perform at least one command, so that every program terminates or reaches its next command in finite time. The sketch follows §4: `iter` is the loop, `converse` is a loop whose state is the dialogue, and `replay` rejects a round that performs nothing.

Apfelmus (2010); Kiselyov and Ishii (2015) define the same type again: Program `instr a` with `Then` and `Return`, and `FFree f a` with `Impure` and `Pure`. The name Program has this precedent. Both warn that left-nested bind is quadratic; Kiselyov and Ishii’s remedy keeps the continuation as a type-aligned queue instead of a composed function. That is not pressing here, since `converse` binds to the right and every operation is a model call or a command. Their open unions, with one interpreter per effect, are the alternative to a closed signature such as `Op`, and are worth the machinery only when others must add operations. Their `feedAll`, which answers requests from a list, is `replay` for a single operation, except that it fails at the end of the list where `replay` returns the pending call. On scopes they stop: “the detailed investigation of the notion of scope deserves its own paper.”

Xia et al. (2020) give the coinductive version, interaction trees: `Ret`, `Vis e k` and a silent `Tau`, so that a program that loops without performing is still a value. They note that an inductive tree without `Tau` restricts systems to acyclic ones, and that splitting events into a type of operations and a family of answer types, as `Op` does, is McBride’s “general monad”. They use the word *driver* as the sketch does, for the outermost loop outside the proof assistant. Three things apply. Their trace type (§7) is the log: a list of events with answers that ends in a result or in an event “waiting for a response from the environment”. It gives `replay` a specification: done when the log is a complete trace of the program, ask when the log followed by the pending call is one, mismatch when the log is a prefix of none. Their criterion for an interpreter is that it “commutes with `ret` and `bind`”. `replay` returns its cursor, a position in the log with the unread notices and the call counter, so it is a state monad over the cursor and the law can be stated. The sketch’s `iter` is their loop combinator. They also give McBride’s recursion-as-an-effect with a fuel semantics; for `replay` the finite log is the fuel. In their related work, CertiKOS gives a sequential component as “a deterministic function from environment interactions (as represented by the log) to its next action”, which is the type of `next`.

McBride (2015) has the same tree as General `S T X`, with `S` the requests and `T` s the responses: “request-response trees”. His subject is recursion. A recursive function is written as one total step in which every recursive call is a request, and how to run it is chosen afterwards: with a supply of fuel (“petrol”), coinductively, or with a domain predicate. The sketch’s round is such a step, with the loop as a constructor in place of a call request. Every round performs a `sample`, so against a finite log the number of rounds is bounded by the length of the log, and `replay` needs no other fuel. A tool is called in the same way: the sketch’s `call` is a request that names the tool and holds no body, so it is data, and the log holds it as the opening of the call. The interpreter answers it by running the tool the run has under that name; how deep calls nest is bounded by fuel, the length of the log again, since a body is entered only once its opening is logged.

Piróg and Gibbons (2014) study the monad of trees that may be infinite, and show that a guarded recursive equation, one in which every step performs an operation before it recurses, has a unique solution. That is the semantic justification for a looping `converse`. Read the other way, `next` is such an equation with logs as its states: from a log it gives the outcome, or an operation whose answers lead to longer logs. The paper assumes the infinite trees exist and does not say how to obtain them in a proof assistant.

Handlers

Plotkin and Power (2003) say what makes an operation algebraic: it commutes with evaluation contexts, $E[\text{op}(N, x.M)] \equiv \text{op}(N, x.E[M])$. The perform clause of `bind` is this equation. Their generic effect, $\text{gen } M \equiv \text{op}(M, x.x)$, is the sketch’s `perform op`, which is `.perform op .pure`. They also show (Example 3) that exception handling fails the equation and is “of a different computational character”, a deconstructor of effects. A construct that runs a body in a frame of its own is of that kind too: `bind` would extend its continuation but not its body. The sketch keeps it out of the syntax: `call` names a tool and holds no body, so it is an operation like `perform`, and what is scoped about it is in the interpreter, which runs the named tool in a child frame. Raising is algebraic as well, and the sketch’s `fail` is that; `try` and `catch` are the other half, written as a function of the whole program (`attempt`).

Plotkin and Pretnar (2009) make a handler a model of the effect theory, and handling the unique homomorphism out of the free model. The pure and perform clauses of `replay` have this form. Their input-redirection handler (§6.7), which answers `in` from the head of a list passed along as a parameter and performs the `real` in once the list is empty, is the nearest construct to `replay` on this side of the literature. It has no check that the answer belongs to the operation, and it performs where `replay` stops. Their handling construct, `try t with H as x in t'`, is what the interpreter does for a `call`: it handles the body of the tool with one fixed handler, which puts the body’s events in a child frame, and goes on with the continuation. Their correctness condition, that a handler respect the equations of the theory, is vacuous while `Op` has no equations, and becomes an obligation once one is asserted. They warn that parallel composition “apparently requires a binary variant of handlers”, which bears on running tool calls concurrently.

Scopes

Wu, Schrijvers and Hinze (2014) show that when a handler also delimits a scope, the order of handlers and the extent of the scope cannot be chosen independently. They give two remedies: `begin` and `end` markers in the syntax (§7), and higher-order syntax in which the operation carries the computation (§10), as in `Catch (m x) (e -> m x) (x -> m a)`. `call tool k` is the second with the computation replaced by its name: the body is the tool the run has under that name, and `bind` extends only the continuation, as their `emap` does. They conclude that higher-order syntax is “strictly more expressive” and that markers “run the risk of being unbalanced”. The log has both: every event carries its whole frame path, and a call is bracketed by an opened event, which is the call, and a returned event, which holds what the tool gave. `replay` knows both in advance and checks the logged ones against them, so the markers cannot be out of place, and a tool that performs nothing still shows. A scope that means more than a frame is their `Catch`, a scope with a handler argument, and a `call` is one: it catches the failure of its tool and gives its continuation the value or the error, and the log marks which. Their example poses the question such a design faces, whether the effects inside a failed scope are kept or rolled back; for an append-only log they are kept. A stopped event from outside is a failure that nothing in the agent catches: it ends every frame of the agent, and the run goes on with what follows. Timeouts and joining parallel siblings are not in the paper.

Piróg, Schrijvers, Wu and Jaskelioff (2018) give the formal account. In their monad of scoped syntax a scoped operation is a pair of the computation in the scope and a continuation, with the type between them hidden. Program is not this monad: a `call` holds the name of its computation and not the computation, so the two are paired by the interpreter. What a run leaves is their flat form all the same. Their Theorem 5.1 shows that the nested syntax is isomorphic to a flat one with matched opening and closing brackets, which justifies storing a run as a flat log; the sketch’s opened and returned events are those brackets. An interpreter of scoped syntax is a *scoped algebra*: besides the operations it says how to enter a scope (`promote`) and how to leave one (`demote`), at every depth. The `call` clause of `replay` fits as a state-passing instance: enter with the child frame and a counter of zero, leave by continuing the parent with its counter plus one. Their index is the depth of nesting only; a path such as `Frame`, and several kinds of scope, are future work there.

Bach Poulsen and van der Rest (2023) generalise to operations with any number of computation parameters (hefty trees), whose bind extends “continuations (but not computation parameters)”, as `Program.bind` does for the rounds of a loop. Meaning comes in two stages: an *elaboration* turns each higher-order operation into ordinary operations and handlers, and handlers then interpret the result. Laws are stated and proved for an elaboration together with a handler, not for the syntax. For a `call`, the elaboration that matches the log is a handler applied to the tool’s body that puts its events in a child frame, and its laws hold only up to renumbering, since a tool that does nothing still consumes an ordinal and logs its brackets. Their two elaborations of `catch`, one keeping the effects of the failed block and one rolling them back, are the choice for a failed call; the sketch keeps them. Their `spawn` elaborates to an interleaving, which strict in-order matching would not accept.

Replay

Koppel, Scherer and Solar-Lezama (2018) obtain a continuation by “replaying the entire computation from the start, guiding it using a recording”. Their past and future stacks are the consumed and the remaining log, and the replayed computation is constant, like the program given to `next`. Their Theorem A.5 (a replayed configuration reduces back to the original by pure steps) and Lemma A.3 (appending to the future preserves those steps) are the two theorems the sketch should have. The difference is why replay is needed. They replay because a direct-style language cannot hand over the continuation, and say that languages with monad syntax “may not benefit”. The sketch already holds the continuation, as `k` in `perform op k`, and replays only because `k` is not persisted. So a live driver should continue with `k` and `replay` only after a restart, which is their optimisation of §6.2 and removes the quadratic cost. They require that the replayed computation have no other side effects but do not check it; mismatch is the check. Of persisted logs they say only that `replay` logs in web frameworks “are equivalent to serializing a continuation”, which is the architecture in one sentence.

Burckhardt et al. (2021) formalise Durable Functions. An orchestration is deterministic code that is replayed; nondeterminism and I/O “must be wrapped in an activity, to ensure it is properly recorded and replayed”. So orchestration, history and activity are program, log, and `sample` or `exec`. Their main result (Theorem 6.4) is that persisting histories is bisimilar to persisting intermediate states, with determinism isolated in one lemma: a fault-free reference model and a proof that the implementation simulates it, the form in which to state the sketch’s correctness. They differ from the sketch in four ways. The history records a request when it is issued and its response when it arrives, as two entries matched by a fresh identifier, not by position, which is what entries fan-out and fan-in. Replay happens once per work item and is followed by live execution, not once per operation. History growth is named as a problem, with two remedies: sub-orchestrations with histories of their own, and `continue_as_new`, which restarts with a value and replaces the history by one entry; compacting the dialogue is the natural point for it in an agent. And the crash window is closed only for internal state, by committing the new history atomically with the input it consumed: “external calls can be duplicated if there are multiple attempts at executing a work item.” `exec` needs its own answer: the sketch runs a command on the version of the workspace the log has reached, so that a command run again after a crash starts from the same workspace. The paper has no runtime check for nondeterminism, no versioning of code against existing histories, and no messages delivered into a running orchestration, so nothing corresponding to `inbox`.

Thiemann (2002) is the earliest instance found of the whole scheme. A WASH/CGI script is re-run from the start on every request, with “a log of the responses from the user and from I/O operations”, and “each input operation tries to get its result from the front of the first queue. Only if the queue is empty it performs the actual input.” The justification is the sketch’s: in a pure language “all other values can be recomputed with equal outcomes”. The log holds answers only, with no record of which operation each one answers, so the check behind `mismatch` is an addition of the sketch. The log travels in the page, so every page is a prefix that can be continued, which is forking a run. For state outside the program he uses time-stamped handles that detect a value that is no longer current; the versions of the workspace in the sketch’s log are such handles. His script stops at a form until the user responds. The sketch’s `ask_user` stops the same way, at a read of the `inbox` that waits, and the reply is a notice to the call that asked, logged when the person gives it.

Zhang et al. (2020) built Beldi, which re-executes a crashed function and skips the operations already logged under an instance identifier and a step number. The authors separate cases the sketch treats alike. A read need not be atomic with its log entry, since a lost answer costs only a second read; that covers `sample` and `time`. A write must be atomic with its log entry, which Beldi achieves with one conditional write in the database that also holds the log. A call to another function is made exactly-once by logging the callee’s identifier first and having the callee deduplicate on it. For anything that does not cooperate, Beldi “only guarantees that the operation is performed at least once”. So `exec` is exactly-once only if the executor records an identifier and the output atomically with the change to the workspace. Otherwise the guarantee to state is at-least-once. The sketch states it so: a command whose answer was not logged is run again, on the version of the workspace the log has reached. It writes no intent before a command, so on restart a partial effect of the command cannot be told from a change made from outside while the driver was down, and the driver restores the logged version.

Goldstein et al. (2020) built Ambrosia, which also replays a log of inputs. They treat input that is pushed from outside and cannot be replayed, such as a person typing. For such an *impulse* “the arguments are recorded in the log before execution”. The sketch’s `arrived` is this: the driver appends a notice when it arrives, and that is the only place the notice is held. A read of the `inbox` is logged as a mark that holds the positions of the notices it took, so which read took a notice can be seen in the log without the program. To bound the log they checkpoint, which presupposes state that can be serialised; in the sketch that is the dialogue at the top of a round, not a continuation. Their upgrade recovers with the old code and converts its state at a checkpoint. It is the one treatment of code versions found, and it avoids replaying an old log against new code.

Concurrency

The sketch is sequential by decision: one call at a time, matched by position. These three papers say what parallel calls would take.

Gu et al. (2018) is the source of the CertiKOS sentence above. All participants append events to one global log; a participant’s “strategy φ_i is a deterministic partial function from the current log l to its next move”; and *replay functions* “reconstruct the current shared state from

the log”. The events of others are not skipped. A thread is given its meaning relative to an environment context E , and the machine “repeatedly asks E for environmental events until the control is transferred to i ”. For parallel calls this is the model: when one frame is replayed, events of other frames are environment events at each perform, not a mismatch, and what participants may assume of each other is an invariant over the log. Notices are already taken so: replay passes them on its way to the next answer and keeps them for the inbox. The correspondence is partial. Their replay rebuilds shared state, not a continuation; an event is the participant’s own move, with results recomputed and not stored; and there is no nesting.

Marlow et al. (2014) built Haxl, in which a monadic program exposes every request it is blocked on. A blocked result carries a sequence of requests and the rest of the computation; `<*>` explores both of its arguments and joins their requests, where `>=>` cannot look past its first. They considered and set aside the constructor `Blocked (Request r) (r -> Fetch a)`, which is `perform`, because with several requests the results are hard to connect to their continuations, and used mutable references. A pure replay has a key instead: the frame and an ordinal within it. Their soundness argument needs requests to be reads, since “operations in `Fetch` may take place in any order”. Two execs in one workspace are the excluded case, and their cache keyed by request would be wrong for `exec`. Their `<*>` differs from the one that `bind` defines, which they concede breaks a convention; an explicit parallel constructor avoids the question.

Chappe et al. (2023) add nondeterministic choice nodes to interaction trees. Parallel composition there is not a fold over the tree. It needs a function head that computes the actions a tree can take next, which is what a `next` returning a set of calls would be; and a scheduler is a *refinement* that resolves the choices, of which a recorded log is one. Two limits matter: nothing joins the results of parallel branches, and a question and its answer are one step, so several outstanding requests are not modelled.

Names and reuse

Wingate, Stuhlmüller and Goodman (2011) name each random choice by its “structural position in the execution trace” (the functions, line numbers and loop indices on the call stack), so that a re-execution whose control flow has changed can look up and reuse the choices that are unaffected. Naming choices by order of encounter they call naive: after one change the rest of the sequence is misaligned and “must probably be discarded”. The sketch matches by order, and `mismatch` is that misalignment. This costs nothing today, because a program is replayed only against its own log and its control flow cannot change. Frame paths are structural across nesting but ordinal among siblings, and they name calls, not operations. The paper’s scheme becomes necessary in two cases. If sibling calls run in parallel, their events interleave and answers must be found by name, which needs a name per operation: the frame path and a counter within the frame. If an old log is replayed against an edited program, their update rule is the template: reuse an answer when the name matches and the parameters are equal, ask again otherwise, and drop what is stale. Names stable under edits need a static component, such as the tool-call identifier, in place of ordinals; the authors claim no optimality for their scheme.

Hammer et al. (2015) separate two ways of matching a computation against an earlier one: *structurally*, by content, and *nominally*, by a first-class name. Their example is an insertion into a list: everything whose content transitively includes the new element fails to match, although little changed. A `sample` request that carries the whole dialogue is matched structurally, so one early change invalidates every later request. Names given by position do no better: they “identify each extension by a global count”, which shifts. Their prescription is to derive names from identities in the data, by forking the name of the input, and to keep identity apart from validity: find the earlier result by name, reuse it if the argument is equal, otherwise recompute and dirty what depends on it. A name used twice in one run is an error they detect, and a poor choice of names costs reuse but never correctness.

Mokhov, Mitchell and Peyton Jones (2018) classify build systems by what they remember. A *verifying trace* records the hashes of the dependencies used last time; a *constructive trace* “additionally stores the resulting value”. An event of the log is constructive in content but flat, and for them a failed verification means rebuild and carry on, where `mismatch` stops. The system nearest the sketch is one they place outside their framework: Fabricate, “a script that is run in-order”, later run “from start to finish, skipping any commands where no inputs have changed”, which they can encode only by assuming a linear dependency between successive commands. That assumption is the sketch’s strict order, and it, not the edit itself, is why nothing after the first changed operation is reused. They also name what cannot be cached: volatile tasks, here `time`, and tasks with untracked dependencies, as `exec` would be if the versions of the workspace were not in the log. Re-running from the start for each missing value is their restarting scheduler; holding the continuation is the suspending one.

Cusumano-Towner et al. (2019) built Gen, which keeps a trace as a map from hierarchical addresses to values, where “the address hierarchy mirrors the call hierarchy” and the user chooses the address at each call. `update` takes a trace, new arguments and some new values, and returns a new trace together with a *discard*: the values that were overwritten or are no longer visited. That is the operation to put in place of `mismatch`, and it needs events keyed by name. It is defined for new arguments to the same program, not for an edited program. Their `Unfold` combinator fits a loop such as `converse`, but the saving is limited to the unchanged prefix, because each round’s state contains the whole dialogue.

Model calls as random choices

Bingham et al. (2019) say of Pyro, in this short paper, only that a program’s `sample` statements are primitives “whose behavior is overridden by inference algorithms”, and that this is done by Poutine, a library of effect handlers. The handlers themselves, including any that trace or replay, are not described in it.

Dohan et al. (2022) define a cascade as “a probabilistic program that includes string-valued random variables, sampled from an LM”. An agent program whose control flow depends on its samples is a cascade, and a log is one trace of it. They recast chain of thought, verifiers, self-consistency and tool use in these terms, and use only forward sampling and rejection. These patterns become available to the sketch once a prefix of a log can be continued several times: sample several continuations and vote, reject and resample, or rank with a verifier. The paper says nothing about tools with side effects when a program is run again, so the requirement is the sketch’s to state: each continuation needs its own workspace from the point of the fork.

What the sketch takes

- The loop is a constructor with a state and a guard, so `converse` and `replay` are total and the log is the fuel (Hancock and Setzer §4; Xia et al. §4; McBride; Piróg and Gibbons).
- `replay` returns its cursor, and a log that is no trace of the program is a result of its own, `mismatch`, not an outcome of the agent (Xia et al. §7).
- A command happens at least once, and runs on the version of the workspace the log has reached, which the driver restores on restart (Zhang et al.; Burckhardt et al.).
- What happens unasked is logged when it arrives, once. A read of the inbox is logged as a mark with the positions of the notices it took (Goldstein et al.; Gu et al.).
- The log records the version of the workspace that each command and each change from outside leaves, from the root on: the workspace that someone provides, which is the first entry of the log (Thiemann; Mokhov et al.).
- A tool is called by its name. The call is data, a request that the interpreter answers by running the tool the run has under that name in a child frame, and the log brackets it: an opening, which is the call, and a return with what it gave (McBride; Piróg et al.; Wu et al. §7, §10).
- A tool that asks a person is called with the question and waits for a reply: a notice to the call that asked, of the form the question asks for, which is checked when it is appended. The driver stops until then, and the same loop resumes the run (Thiemann; Goldstein et al.).
- A stop from outside ends every frame of the agent, and the run goes on with what follows it: the graders, whose effects the agent cannot see. A grader is a tool that runs an external program, an operation: a command in a fresh container of a pinned image, on a checkout of the workspace, which the workspace of the run does not follow. A state of a run is a prefix of its log, and is graded on a fork stopped there, one for each version of the workspace (Wu et al. §10; Mokhov et al.).
- A program can fail. A call catches the failure of its tool, the log marks it, and what the tool did is kept; `try` and `catch` elsewhere are a function of the program and leave no mark. The answer to an operation may be an error, logged like any answer, and the operation then fails (Plotkin and Power; Wu et al. §10; Bach Poulsen and van der Rest).

What it leaves

- The two theorems are checked on a run, at every prefix of its log, and not proved: `replay` agrees with the trace specification, and appending the answer to the call `next` returned continues the program where it stopped (Xia et al. §7; Koppel et al. A.3, A.5; Burckhardt et al. 6.4).
- The driver replays from the start for every call. A live driver would keep the continuation and replay only on restart; and since the state of a loop is data, replay could start at the last round, and the log be cut there (Koppel et al. §6.2; Burckhardt et al. §3.2, §6; Goldstein et al.).
- A failed call is not rolled back, and one sub-agent cannot be stopped from outside. A timeout on a call and a stop that names a frame would be further scoped operations, with laws stated for an elaboration (Bach Poulsen and van der Rest; Wu et al. §10).
- Matching is by position, so an edited program reuses nothing after its first changed operation. Names from identities in the data, and a discard in place of `mismatch`, would (Hammer et al.; Cusumano-Towner et al.; Mokhov et al.; Wingate et al.).
- Forking a run from a prefix of its log, with a workspace for each fork (Thiemann; Dohan et al.).
- An intent logged before a command. Without it a command that may have run leaves no trace: an effect beyond the workspace can happen twice unrecorded, and a change made from outside while the driver was down is lost on restart (Zhang et al.).
- An event holds the call it answers, which serves only the check behind `mismatch`. For a sample that is the whole dialogue, so the log grows quadratically; a hash of the call would serve.
- Parallel calls, by decision (Marlow et al.; Chappe et al.; Gu et al.).

Two questions stay open after all twenty-five. Replaying an old log against an edited program is treated only by analogy: Gen updates a trace for new arguments, Hammer et al. for edited inputs, and Goldstein et al. avoid the case. And none gives exactly-once execution of a command on a target that does not cooperate; Zhang et al. and Goldstein et al. both fall back to at-least-once.

Papers

P. Hancock, A. Setzer. Interactive programs in dependent type theory. CSL 2000.
H. Apfelmus. The operational monad tutorial. The Monad.Reader 15, 2010.
O. Kiselyov, H. Ishii. Freer monads, more extensible effects. Haskell 2015.
L. Xia et al. Interaction trees: representing recursive and impure programs in Coq. POPL 2020.
G. Plotkin, J. Power. Algebraic operations and generic effects. Applied Categorical Structures 11, 2003.
G. Plotkin, M. Pretnar. Handlers of algebraic effects. ESOP 2009.
N. Wu, T. Schrijvers, R. Hinze. Effect handlers in scope. Haskell 2014.
J. Koppel, G. Scherer, A. Solar-Lezama. Capturing the future by replaying the past. ICFP 2018.
S. Burckhardt et al. Durable Functions: semantics for stateful serverless. OOPSLA 2021.
D. Wingate, A. Stuhlmüller, N. Goodman. Lightweight implementations of probabilistic programming languages via transformational compilation. AISTATS 2011.
C. McBride. Turing-completeness totally free. MPC 2015.
M. Piróg, J. Gibbons. The coinductive resumption monad. MFPS 2014.
M. Piróg, T. Schrijvers, N. Wu, M. Jaskelioff. Syntax and semantics for operations with scopes. LICS 2018.
C. Bach Poulsen, C. van der Rest. Hefty algebras: modular elaboration of higher-order algebraic effects. POPL 2023.
P. Thiemann. WASH/CGI: server-side web scripting with sessions and typed, compositional forms. PADL 2002.
H. Zhang et al. Fault-tolerant and transactional stateful serverless workflows. OSDI 2020.
J. Goldstein et al. A.M.B.R.O.S.I.A: providing performant virtual resiliency for distributed applications. VLDB 2020.
R. Gu et al. Certified concurrent abstraction layers. PLDI 2018.
S. Marlow, L. Brandy, J. Coens, J. Purdy. There is no fork: an abstraction for efficient, concurrent, and concise data access. ICFP 2014.
N. Chappe et al. Choice trees: representing nondeterministic, recursive, and impure programs in Coq. POPL 2023.
M. Hammer et al. Incremental computation with names. OOPSLA 2015.
A. Mokhov, N. Mitchell, S. Peyton Jones. Build systems à la carte. ICFP 2018.
M. Cusumano-Towner, F. Saad, A. Lew, V. Mansinghka. Gen: a general-purpose probabilistic programming system with programmable inference. PLDI 2019.
E. Bingham et al. Pyro: deep universal probabilistic programming. JMLR 20, 2019.
D. Dohan et al. Language model cascades. 2022.